ENGINEERING / LOCAL DEVELOPMENT

DinoStudio Developer Guide

How to run the application, understand its service boundaries, analyze repositories, and prepare local model training.

Audience Frontend, API, and local ML developers

Scope Current workspace implementation as of November 2026

Source of truth Project code and requirements files; this guide is not a GPU training validation report.

Contents

01 / Quick start

02 / Architecture and rationale

03 / How to use each view

04 / Scraper and training data flow

05 / Project structure

06 / Dependencies and configuration

07 / API map

08 / Extending the codebase

09 / Troubleshooting and limits

01 / Quick start

Run commands from the repository root on Windows PowerShell. Node.js/npm and Python 3.11+ are required for the scraper. The Angular app and the scraper are **different processes**. Training needs an additional compatible Python environment, but not a permanently running third server.

Install dependencies once

```
npm install
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-scraper.txt
```

Use a supported installed Python version if `py -3.11` is unavailable. The Node service also uses `.venv` by default for its Python-backed tools; install the corresponding optional requirements only when needed.

Terminal A: Angular + Node service

```
npm start
```

The launcher chooses an available Angular port from 4300 to 4319 and prints the URL. It starts Angular and the local Node terminal/notebook service. Keep this terminal open. **It does not start the scraper FastAPI service.**

Terminal B: scraper + training API

```
$env:DINO_APP_PORT = "4300"             # replace with Terminal A's actual port
$env:SCRAPER_ALLOWED_ROOTS = "C:\Projects;D:\Work"  # for local-path registration
.\.venv\Scripts\python.exe -m scraper_api.run
```

The API binds to `127.0.0.1` and prints its URL. It tries port 8000 and chooses the next available port if needed. Do not start multiple copies. The browser discovers the scraper API on ports 8000-8020.

```
Invoke-RestMethod http://127.0.0.1:8001/api/health
# Replace 8001 with the port printed by Terminal B.
```

> **Where to go.** Open the URL printed by Terminal A: `/` is Canvas, `/scraper` is Repository scraper, and `/training` is Training. A scraper-only workflow needs Terminals A and B; the separate GPU Python is needed only for a training run.

Optional: configure training before starting Terminal B

```
$env:TRAINING_PYTHON = "C:\venvs\dino-training\Scripts\python.exe"
# Restart Terminal B after changing the variable.
Invoke-RestMethod http://127.0.0.1:8001/api/training/preflight
```

`TRAINING_PYTHON` must be an absolute executable path in a *separate* CUDA-enabled environment, not the API's `.venv`. Preflight returns reasons when it is blocked. A 200 response does not imply `ready: true`. Existing scraper jobs are in memory and disappear when Terminal B restarts; downloaded results ZIPs remain usable.

02 / Architecture and rationale

```
Browser / Angular app  (localhost:4300-4319)
  |  Canvas tools           |  Scraper + Training pages
  v                         v
Node local service           FastAPI scraper API  (127.0.0.1:8000-8020)
(127.0.0.1:appPort+1*)      |  scan / parse / match / check / export
  |                          |  in-memory jobs + ZIP artifacts
  +-- WSL terminal           v
  +-- notebook Python       runs/<id>/  +  TRAINING_PYTHON subprocess
  +-- code/image Python                    |  Unsloth + TRL LoRA training
  +-- local Ollama HTTP                    |  held-out evaluation + GGUF export
                                           +-- lazy llama-cpp GGUF worker

* Node service retries subsequent ports if its first port is busy.
```

Boundary	Responsibility	Why separate?
Angular	Canvas, file picking and ZIP creation, analysis tables, training controls, polling and SSE log display.	Responsive interaction; repository code is never executed by the scraper UI.
Node local service	Notebook sessions, WSL terminal, data viewer, code/image workers, Ollama requests.	Hosts the canvas tools and their short-lived processes independently of FastAPI.
FastAPI + scraper core	Restricted project intake, in-memory job lifecycle, local parsing/checking, export and artifact endpoints.	Core analysis is Python without importing training GPU libraries.
Separate training Python	Dataset validation when configured, CUDA/VRAM checks, LoRA worker, evaluation, GGUF export, llama-cpp generation.	Isolates Torch/Unsloth version constraints and expensive GPU work from API request handling.

Node and FastAPI both have health endpoints, but they are not interchangeable. Angular's `local-api.ts` discovers Node on ports 4301-4320; `scraper-api.service.ts` discovers FastAPI on 8000-8020. FastAPI CORS must allow the Angular origin. Only one training/evaluation run uses the GPU at a time; cancelling terminates its process group.

03 / How to use each view

Canvas: compose local tools

The canvas is built with Foblex Flow. Create and connect terminal, notebook, GitHub project, Ollama, text-to-image, code-generator, and data-table nodes. Folder tree is browser-side. The terminal node accepts an absolute Windows folder, translates it into WSL, and executes commands there; use only trusted folders and commands. Notebook execution runs selected Python code with the current user's permissions.

Node handlers enforce request size/time limits. Ollama requests go to local `127.0.0.1:11434`. Code and image model downloads are optional and separate from the scraper/training service. The data viewer uses pandas to read CSV/XLSX/JSON.

Repository scraper: produce reviewable artifacts

1. Go to `/scraper`. Choose a folder (browser creates and uploads a ZIP), choose individual files, or register an allowed local path. For a path, configure `SCRAPER_ALLOWED_ROOTS` in Terminal B.
2. Adjust analysis config only when needed: ignored files, extensions, Git history, held-out test features, model tokenizer, token limit, task types and mutations.
3. Click **Run analysis**. Status and activity log show scanning, parsing, matching, checks, token counting, secret scan and export. Files, Endpoints, Calls, DTO pairs, Matches, Issues, Examples and Coverage are paged tables with search/sort.
4. Parsed tables become available before export finishes. **Download results** requires a successfully completed job; a failed export can still leave parsed tables visible.

Training: preview, run, evaluate, activate

1. At `/training`, choose a completed scraper job from the same API session or upload a **Download results ZIP**. A source-code ZIP is not training input.
2. Review model/LoRA config and preflight. **Preview dataset** checks the secret marker, exact chat schema, train/test feature separation, and real tokenizer lengths. The API interpreter can preview a locally cached tokenizer offline when no training Python is configured; this does not authorize training.
3. After preflight reports `ready: true` and preview passes, click **Start training**. Runs lists status and metrics. Run detail polls progress and streams loss/log events; cancel stops the subprocess tree. A dry run uses the configured 0.5B model for two steps.
4. Evaluate a completed adapter. The report compares base and fine-tuned outputs on the held-out split by task. Only a real run with GGUF and completed evaluation can be activated.
5. Use **Try it** with the active model. The same saved system message used for training is sent to the lazy local llama-cpp worker.

> **No GPU environment yet?** Scraper and tables still work. Training preflight remains blocked until a separate compatible CUDA Python is configured. GGUF conversion and the Unsloth/TRL/llama-cpp combination have not been verified on this Windows workspace; the repository has no pinned `requirements-training.txt`.

04 / Scraper and training data flow

The scanner gathers allowed files; C# and TypeScript parsers produce classes, controllers, models and service calls. Match and check modules identify endpoint/call pairs, DTO pairs and issues. Task builders create chat examples; export performs real tokenizer counts and a secret scan before writing the ZIP.

```
Repository / source ZIP
  -> scan -> parse C#/TS -> match + consistency checks
  -> task examples -> tokenize + secret scan -> results ZIP
  -> dataset validation + feature holdout -> LoRA adapter
  -> held-out base-vs-tuned evaluation -> GGUF + model card
  -> activate model -> local chat generation
```

The results ZIP contains `train.jsonl`, `test.jsonl`, `pairs_all.jsonl`, per-task `pairs_*.jsonl`, `chunks.jsonl`, `project_model.json`, `consistency_report.csv` and `manifest.json`. A JSONL row is chat-format:

```
{"messages":[{"role":"system","content":"..."},
             {"role":"user","content":"..."},
             {"role":"assistant","content":"..."}],
 "meta":{"id":"...","task_type":"dto_to_ts","feature":"...",
         "source":"...","split":"train"}}
```

A new successful export sets `manifest.secret_scan_passed: true`. The training loader rejects old ZIPs without a confirmed pass, secret-scan failures, malformed messages, duplicate IDs, feature leakage, or empty train/test splits. Rows over the configured sequence length are flagged and removed. Holdout is by *feature*, not random rows; two isolated service files may not provide enough train/test examples.

FastAPI keeps scraper jobs in memory; their ZIP downloads are lost on API restart. Training runs persist under ignored `runs/<uuid>/`: `dataset.zip`, `config.json`, `status.json`, `progress.jsonl`, `worker.log`, `metrics.json`, `adapter/`, and (for real runs) `gguf/` and `model_card.json`. Evaluation adds `eval_progress.jsonl`, `eval.log` and `eval_report.json`. Model downloads are bundled in that run directory.

05 / Project structure

```
dinoStudio/
  src/app/
    app.routes.ts, app.html        application routes and navigation
    graph/canvas/canvas/           Foblex graph and tool nodes
    folder-tree/                   browser folder UI
    local-api.ts                   Node local-service discovery
    scraper/data-access/, ui/      ZIP preparation, API client, result tables
    training/data-access/, ui/     typed training client, SSE, five views
  scripts/
    dev.mjs, terminal-server.mjs   Angular + Node launcher and API
    notebook-session.py, ...       isolated notebook/data/model helpers
  scraper_api/
    run.py                         loopback port selection and Uvicorn
    app/api/                       FastAPI routes, workspaces, job managers
    app/core/                      scan, parse, match, checks, tasks, export
    app/training/                  config, dataset, preflight, train, evaluate,
                                   export, runner, registry, serve
  runs/                            generated local training artifacts (ignored)
  requirements-*.txt               optional Python dependency groups
  clenupdata.ipynb                 notebook specification for scraper + LoRA
```

Keep parsing and scoring rules in `app/core/` and training work in `app/training/`; neither should depend on FastAPI request objects. HTTP validation, response codes and streaming belong in `app/api/main.py`. Angular service files own typed requests; page components own UI signals, not backend model execution.

06 / Dependencies and configuration

Layer	Key packages / prerequisites	Install / when needed
Angular + Node	Angular 22, Material/CDK, Foblex Flow, RxJS, JSZip, PrimeNG, AJV; Node/npm.	`npm install` once; `npm start` for the dev launcher.
Scraper Python	FastAPI, Uvicorn, Pydantic, python-multipart, pandas, Transformers.	`pip install -r requirements-scraper.txt` into `.venv`.
Canvas Python	Notebook: nbclient, nbformat, ipykernel, pandas/openpyxl. Code: Torch, Transformers, Accelerate. Image: Torch, Diffusers, Pillow.	Install the relevant `requirements-notebook.txt`, `requirements-code.txt`, or `requirements-image.txt` only for tools used.
Training Python	Separate CUDA-capable environment; Torch, Unsloth, TRL, Datasets, PEFT, Accelerate, bitsandbytes, Transformers and llama-cpp-python.	`TRAINING_PYTHON` absolute path. Compatible versions and GPU runtime remain unverified; no pinned training requirements file exists.
Optional external tools	WSL/Linux distribution for terminal node; Ollama and a local model for Ollama node; cached tokenizer/model files when operating offline.	Only needed when using that feature.

Setting	Purpose
`DINO_APP_PORT`, `DINO_APP_ORIGINS`	Angular origin(s) allowed by scraper FastAPI. Set the actual port from `npm start` before starting Python; origins override accepts a comma-separated list.
`SCRAPER_API_PORT`, `SCRAPER_API_PORT_MAX`	API's initial loopback port (default 8000) and end of port search range.
`SCRAPER_ALLOWED_ROOTS`, `SCRAPER_STORAGE_DIR`	Semicolon-separated Windows roots for local-path registration; ZIP extraction storage (otherwise OS temporary directory).
`DINO_API_PORT`, `NOTEBOOK_PYTHON`	Node service port (launched at app port + 1 initially) and optional interpreter for Node-backed Python helpers.
`TRAINING_PYTHON`	Absolute path to a separate interpreter; required for training, evaluation and GGUF generation. Preview may fall back to API Python for cached-tokenizer inspection only.
`TRAINING_TIMEOUT_SECONDS`, `GENERATION_TIMEOUT_SECONDS`	Worker limit (default 86400s) and inference limit (default 180s).

07 / API map

Node and FastAPI listen on loopback. FastAPI advertises OpenAPI at `/docs` and `/openapi.json`. Use the port printed by each process; the examples below omit its host prefix.

Service	Routes	Use
Node local service	`/api/health`, `/api/terminal/*`, `/api/notebook/*`, `/api/ollama/*`, `/api/dataframe/read`, `/api/image/*`, `/api/code/*`	Canvas integrations. POST requests require `X-Dino-Terminal` and the allowed Angular origin.
FastAPI projects	`POST /api/projects`, `POST /api/projects/local`, `GET /api/uploads/{id}`	ZIP upload or allowed local path; poll ZIP validation/extraction.
FastAPI scraper jobs	`POST/GET /api/jobs`, `GET /api/jobs/{id}`, `GET /api/jobs/{id}/{section}`, `GET /api/jobs/{id}/download`	Start, poll, search/sort/paginate sections, download completed export.
Training setup	`GET /api/training/config/default`, `GET /api/training/preflight`, `POST /api/training/preview`	Defaults, package/CUDA/VRAM gate and tokenizer-backed ZIP preview.
Training lifecycle	`POST/GET /api/training/runs`, `GET/DELETE /api/training/runs/{id}`, `GET /api/training/runs/{id}/logs`, `POST /api/training/runs/{id}/cancel`	Subprocess run, persisted status, SSE progress/logs, cancel and cleanup.
Models	`POST /api/training/runs/{id}/evaluate`, `GET .../{id}/eval`, `GET .../{id}/download`, `POST .../{id}/activate`, `GET /api/training/models`, `GET /api/training/models/active`, `POST /api/generate`	Held-out comparison, GGUF packaging, model activation and chat inference.

Training run creation accepts JSON with a completed scraper `job_id` plus config, or multipart `file` and JSON-string `config`. Run status carries separate `status` and `eval_status`. SSE uses `progress`, `log`, `evaluation` and `eval_log` event names.

08 / Extending the codebase

1. **Add a frontend feature:** place a standalone Angular component and styles under the owning `src/app/` feature; define a typed data-access service; register a route in `app.routes.ts` and a link in `app.html`. Follow signals, OnPush and existing Material/Foblex patterns.
2. **Add a scraper rule or task:** extend parsers, match/check helpers or `core/tasks/`, keep model records in `core/models.py`, and update export and result sections as needed. The scraper must analyze text without executing uploaded repository code.
3. **Add an API contract:** validate HTTP input and response in `app/api/`, keep file work restricted and bounded, and keep pure analysis/training logic outside FastAPI modules.
4. **Change training:** preserve run-local paths, separate interpreter and preflight gate. Save exact config, dataset hash and dependency versions in the model card; do not activate a dry run or an unevaluated adapter.

Use `npx ngc -p tsconfig.app.json --noEmit` for an Angular compile check. `npm run build` is the production bundle; font fetching and configured size budgets may fail independently of TypeScript correctness. The repository also contains test files, but no tests or model runs were executed to produce this guide.

09 / Troubleshooting and known limits

Symptom	Cause / action
Scraper API unavailable or CORS error	Confirm Terminal B is running; inspect its printed port and `/api/health`. Set `DINO_APP_PORT` to the actual Angular port before restarting. Avoid duplicate API processes.
ZIP is still scanning or uploading	Folder enumeration/compression happens in the browser first. Activity log shows file counts, API discovery and upload/extraction; Python has no request to log until the ZIP is sent.
Endpoints or DTO pairs are empty	Endpoints need C# controller actions with HTTP-verb attributes. DTO pairs need C# DTO classes and matching TypeScript models; service files alone do not satisfy either rule.
Training preview returns 422	Use the **Download results** ZIP from a newly completed scraper job, not the uploaded source ZIP. A successful export must include `manifest.secret_scan_passed: true`; do not add it manually.
Tokenizer cannot load during preview	The selected model's exact tokenizer is missing or unavailable. API-Python fallback looks for a cached snapshot offline. This workspace has a cached 0.5B dry-run tokenizer, not necessarily the default 7B tokenizer. Select dry run for its actual tokenizer, or provide/cache the configured 7B tokenizer.
Preflight returned 200 but Start is disabled	Inspect JSON `ready` and `reasons`. Missing `TRAINING_PYTHON`, CUDA, VRAM or packages blocks training even though the endpoint responded successfully.
Active model returns 404	Normal before a real GGUF-producing run completes evaluation and is activated. Dry runs cannot be activated.
Jobs disappear after restarting FastAPI	Scraper workspaces and jobs are in memory. Re-register and rerun, or use a previously downloaded results ZIP. Training run folders under `runs/` persist.
Model export or inference fails	Check `runs/<id>/worker.log` or `serve.log`. The GPU dependency combination and Unsloth GGUF converter availability are not verified; model conversion may attempt external tooling downloads, which is not acceptable under a strict offline policy until reviewed.

> **Trust boundary.** The scraper restricts ZIP traversal, symlinks, size and local-path roots; it does not execute repository code. By contrast, the Canvas terminal and notebook nodes intentionally execute user-selected commands/code. Generated training datasets and models can contain source material; keep `runs/` and downloaded artifacts private. Confirm network policy and package compatibility before production GPU work.

Guide source: `docs/developer-guide.html`. Portable PDF: `docs/developer-guide.pdf`. Project implementation, printed ports and runtime error details remain authoritative if they differ from a static guide.